ARKÆS

ARCHITECTURE AND AESTHETICS

Frontend engineering *with clarity.*

Building scalable interfaces, design systems, and cross-platform products where technical structure and visual refinement work as one.

SAMSUNG RESEARCH INDONESIA
MOBILE · TV · CONNECTED SCREENS

01 · WHY ARKAES EXISTS

Arkaes stands for Architecture and Aesthetics, the two sides of frontend work I care about most.

It exists for a practical reason. My design system work at Samsung sits under a strict NDA. I can describe the approach, but I cannot show a component, a token file, or a line of code. So I built the public version. Arkaes is a design system where I make the same kind of decisions in the open, so anyone reviewing my work can read the source instead of taking my word for it.

ONE LAYER BELOW THE PRODUCT

Most of that work sits one layer below the product. Design tokens, component APIs, navigation models, architecture. I like building that layer because it decides whether the hundredth screen gets the same care as the first.

THE GAP I WORK IN

Most of the frontend work I enjoy sits in that gap between an interface that is technically correct and one that fits the moment it gets used.

EVERY CLAIM LINKS TO THE WORK BEHIND IT

±87%
SmartThings Virtual Home
Tech lead for the monorepo rewrite that merged two product variants. Also ±40% smaller app size and ±65% faster Time To Interactive.

15+
OneUX Lab Design System
Project lead. I set the architecture, component API conventions, accessibility standards, and release process for a design system used across several product teams.

~80%
SmartThings Air Care
Project lead on a legacy modernization. Second load ~80% faster, and a full JavaScript to TypeScript migration done without pausing delivery.

"Architecture meets aesthetics." — ARKAES.DEV

How I *think.*

Most frontend problems get easier once you describe them properly. I spend time there first.

FROM AMBIGUITY TO SHIPPED

01	02	03	04
Understand	*Structure*	*Craft*	*Hand off*
the problem and context	flows · patterns · plans	details users can feel	built for the next engineer

communicating the reasoning at every step

01 Understand

SMARTTHINGS AIR CARE

Before I write code, I want to know what the product needs, what users are doing, and what limits the team is working with. On Air Care that meant reading a legacy codebase we inherited from another team and finding which parts cost us the most debugging time, before I proposed any migration.

~50% FASTER FIRST LOAD · ~80% FASTER SECOND LOAD · FULL JAVASCRIPT TO TYPESCRIPT MIGRATION, NO DELIVERY PAUSE

02 Structure

VIRTUAL HOME FOR TV

I looked for one navigation algorithm that would work on every page. There wasn't one. The project moved once I stopped looking and sorted the pages into three groups instead: structured grids, static scattered layouts, and scattered layouts rendered at runtime. Each group got its own strategy. *One problem I could not solve became three that I could.*

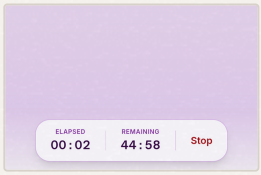
3 LAYOUT MODELS · INDEX MATH FOR GRIDS · DECLARED FOCUS TARGETS · RUNTIME POSITION MAPPING

03 Craft

MILK PUMPING TRACKER

I put the start and stop buttons near the top of the screen. It looked fine in the design file. The person using the app is a mother who may be holding a pump in one hand and the phone in the other, often tired, often in the middle of something else. That was not a layout problem. That was an empathy problem. I moved the controls to a floating position at the bottom, where the hand already rests.

The architecture did not change at all. The app just stopped working against the person using it.



SHIPPED · CONTROLS IN THE THUMB ZONE

04 Hand off

ONEUX LAB
VIRTUAL HOME

I would rather ship something a colleague can extend than something only I understand. That means explicit types, documented component APIs, and a structure that answers "where does this code go?" without a meeting. Versioning through Changesets, accessibility and keyboard reviews, and Storybook documentation so an engineer from another team can understand a component without asking anyone.

±31% QUICKER ONBOARDING · ±70% FASTER BUG RESOLUTION · 15+ CONTRIBUTORS KEPT ALIGNED

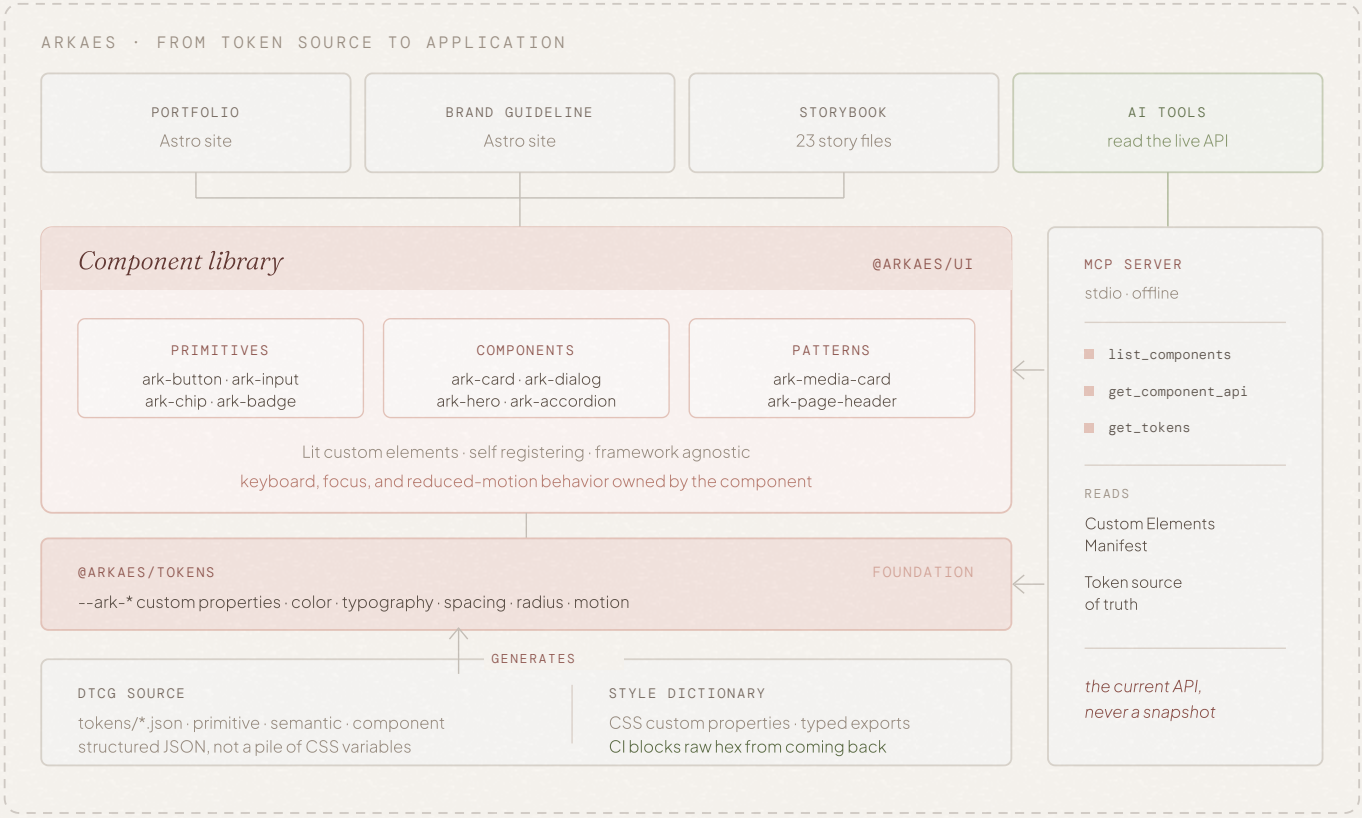
> In code review, technical direction, and mentoring, I explain the tradeoff instead of handing out instructions. Leading a rewrite across more than 10 engineers only worked because the architecture was easy to explain. People route around a structure they do not understand.

Building *systems.*

The system runs on Lit and Web Components, so it stays framework agnostic. It is themed with DTCG design tokens, documented in Storybook, and served through an MCP server so AI tools can read component specifications directly.

This site runs on it. The accordion, buttons, and type on arkaes.dev are the same components documented in the Storybook.

PRIMITIVES	12
COMPONENTS	10
PATTERNS	3
STORY FILES	23
MCP TOOLS	3
TOKEN TIERS	3



Tokens as structured source

Rather than treating design tokens as a collection of CSS variables, Arkaes manages them as structured JSON files following the DTCG format. That generates CSS custom properties and typed exports from one source of truth, and CI now blocks raw hex values from coming back.

Why I replaced RAG with MCP

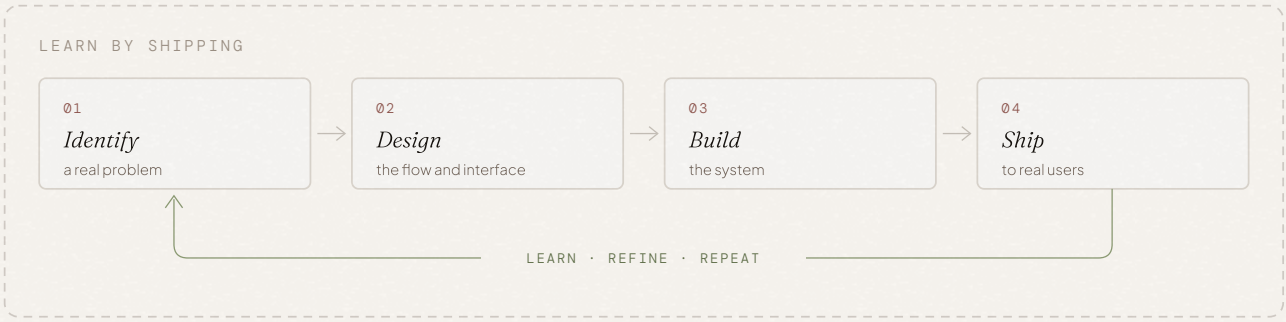
Documentation in a vector database goes stale as soon as the design system changes, and a design system changes all the time. I built an MCP server instead, so AI tools read the current component APIs and design tokens straight from the source.

WHAT THE SYSTEM BUYS

Working without product deadlines lets me think harder about the parts that usually get rushed: what a component API should look like, how tokens should cascade, how a component behaves under keyboard and reduced motion, and how much flexibility is enough before a system gets too complicated.

Learning by *shipping.*

I learn by building something and finding out where I was wrong.



> My degree is in electrical engineering, so I picked up software on my own, through practice. Side projects are how I close that gap. I pick something I do not know yet and ship it, because shipping shows me the parts I only thought I understood.

Arkaes Design System

The long game. A place to keep working on component API design, tokens, accessibility, and documentation without a deadline forcing a shortcut.

LIT · WEB COMPONENTS · STORYBOOK · DTCG · MCP SERVER · ASTRO

Arkhe AI Assistant

Made me design a retrieval pipeline, a streaming response, and safety boundaries that hold up in front of strangers.

ASTRO SSR · OPENAI · SUPABASE PGVECTOR · RAG · LIT

Milk Pumping Tracker

Came from a real family routine and taught me how to let AI support a product without taking it over.

REACT · VITE · FIREBASE · OPENAI API · TYPESCRIPT

Treely

A relationship data model that has to stay correct as a family grows. It also ran on a React Native and NestJS stack I had never shipped before.

REACT NATIVE · EXPO · NESTJS · POSTGRESQL · RN SKIA

> None of them are finished, which is mostly the point. They are where I test ideas before I bring them to work, and sometimes the reverse.

READ THE SOURCE INSTEAD OF TAKING MY WORD FOR IT

PORTFOLIO
arkaes.dev

STORYBOOK
ds.arkaes.dev

SOURCE
github.com/ahmadnaufal-f/arkaes

WRITING
arkaes.dev/blog

"Architecture meets aesthetics."